1 (a) In the context of C++ and using code to illustrate your answers as appropriate:

i What is a destructor?
ii When is a destructor called?
iii How are destructors different from normal member functions?
iv How many destructors can be defined for any one class?
v Why is it needed to write user-defined destructors?
vi Why are objects always destroyed in reverse order of their creation?

[10] in parts [1,2,2,1,2,2]

A destructor is a member function which destroys (destructs or deletes) an object.

A destructor function is automatically called when an object goes out of scope: either because the function ends, the program ends, a block containing local variables ends, or a delete operator is called.

Destructors have the same name as the class but are preceded by a tilde (~). However, they do not take any parameters and do not return anything. For example:

```
class Terminator {
    public:
    string id;

    //Definition for Constructor
    Terminator(string name) {
        id = name ;
        cout << "Constructor called for id: " << id << endl;
    }
    //Definition for Destructor
    ~Terminator() {
        cout << "Destructor called for id: " << id << endl;
    }
};

void backInTime() {
       Terminator arnie3( "T3" ) ;
}// Scope for arnie3(T3) ends here

int main()
  {
    Terminator arnie1( "T1" ) ;
    int i = 0;
    while ( i < 3 )
    {
        Terminator arnie2( "T2" ) ;
        i++;
    } // Scope for arnie2(T2) ends here (three times)
    backInTime() ;
    return 0;
  } // Scope for arnie1(T1) ends here
```

One.

To avoid memory leaks. If a destructor for a class is not defined, then the compiler automatically creates one. This is usually fine, unless a class uses dynamically-allocated memory. When a class member function uses a pointer to dynamically allocated memory, a destructor should be written to release this memory before the class instance is destroyed. Otherwise the memory "leaks", i.e. memory that is no longer needed is not released.

In case the object created later refers to an object created earlier. If the object created earlier has already been destroyed, then this can cause problems.

1 (b) In the context of C++, explain the following terms, using UML and example code to illustrate your answer:

i Abstraction;
ii Encapsulation;
iii Inheritance.

[10] in parts [3,3,4]

Abstraction means only revealing the essential information about data or object, heading the detail. Abstraction can be implemented in C++ using classes. A class can then be used to group data members and member functions, and make those visible or available 'outside' the object via access specifiers (public and private).

Encapsulation is defined as binding together both the data and the functions that manipulate them in one object. It can lead to and support abstraction. For example a class that declares a private variable may only grant access to that variable through use of specific public set() and get() member functions (access functions).

Inheritance is defined as the capability of a class to get properties and characteristics from another class. The sub-class is the class that inherits properties, the class whose properties are inherited is called the super-class.

For example, we could have four types of vehicle: bus, taxi, van and truck. They all have the same properties and functions such as fuelTank and fillTank, but have different properties in the type and limit of load they can carry, e.g. buses and taxis are passenger-carrying, vans and trucks are material-carrying (generally). One class for each would mean defining the common properties four times, but with inheritance, the generic properties only need to be defined once.

Any examples of design/code to illustrate the concepts is acceptable.

1 (c) In the context of C++, explain the following concepts, using code to illustrate your answer:

i Polymorphism (definition only, no code needed);
ii Compile-time polymorphism using function-overloading;
iii Compile-time polymorphism using operator-overloading;
iv Run-time polymorphism using function over-riding.

[10] in parts [1,3,3,3]

Polymorphism is defined as the capability of an object or a function to act in different ways in different situations, or contexts.

Function overloading: multiple functions with the same name but different form. Functions can be overloaded if the function declarations differ from each other by the types and/or the number of arguments in the argument list. It is not possible overload function declarations that differ only by return type.

```
class Wibble {
    public:

    // function with 1 int parameter, returns doubled
    int morph(int x) {
        return int * int ;
    }

    // function with same name but 1 double parameter, returns trunc-ed
    double morph(double x) {
        return trunc( x ) ;
    }

    // function with same name and 2 int parameters, returns both added
    int morph(int x, int y) {
        return x + y ;
    }
};

int main() {

    Wibble bob;

    bob.morph(1);

    bob.morph(3.14);

    bob.morph(2,3);
    return 0 ;
}
```

In the above code, a single function named `morph` acts differently in three different situations which is the essential feature of polymorphism.

Operator overloading: an operator behaves differently depending on the operands.

```
class Set {
    public:

    Set operator + (Set s1, s2) {
        return union( s1, s2 ) ;
    }

    Set operator & (Set s1, s2) {
        return intersection( s1, s2 ) ;
    }

    Set operator - (Set s1, s2) {
        return setdifference( s1, s2 ) ;

    private:

    Set union(Set s1, s2){
       // etc.
    }
    // etc.
};
```

Can now write setX = setY + setZ; instead of setX = union(setY, setZ); and so on.

Function over-riding: a derived (sub) class has a definition for one of the member functions of the base (super) class

```
class Duper {
    public:
    void print() {
        cout << "print super class" << endl;
    }
    void display() {
        cout << "display super class" << endl;
    }
};

class Woofer:public Duper {
    public:
    void display() {
        cout<< "display sub class" << endl ;
}
    void exhibit() {
        cout << "exhibit sub class" << endl ;
    }

};

int main() {
        Duper man ;  // object of super Duper class
        Woofer dog ; // object of sub Woofer class

        man.print() ;
        man.display() ;
        // man.exhibit() ; -- won't work (why not?)

        dog.print() ;  // inheritance
        dog.display() ; // function override
        dog.exhibit() ; // all of sub's own work

        return 0 ;
}
```

2 In C++, a vector is a sequence container representing arrays that can change in size. Useful public member functions are:

push_back //add element at the end of the vector
erase //erase element at an index
at //access element at an index

In C++, a pair is class couples together a pair of values, which may be of different types (`T1` and `T2`). The individual values can be constructed and accessed through its public member functions:

make_pair(*x,y*) // construct a pair object with its first element set to *x* and its second element set to *y*
first //access value of first element of the pair
second //access value of second element of the pair

(a) A standard deck of playing cards consists of 4 suits and 13 face cards, from numbers 1 (ace) to 10, and jack, queen, and king

Declare two constants to represent this information.
[2]

```
const int numSuits = 4; //number of suits in a deck
const int numFaces = 13; //number of cards in each suit
```

(b) Explain (using code) how the following declaration could be used to represent a deck of cards, commenting on the suitability of this representation.

vector <pair<int,int>> Cards ;
[4]

First component of the pair represents the suit (1=hearts, 2=clubs, etc) second the face (1=ace, 2=2,…13=king); vector of 52 such pairs represents the entire pack of cards

```
pair<int,int> card ;
card = make_pair(1,1) ;
Cards.push_back(card) ;
```

Much better to use an enumerated type, or if one really must use integers for the representation, at least use a sub-range of the integer type. However, although the maximum size of the array is fixed, it is expected for it to grow and shrink as cards are dealt and returned, so to have all the dynamic memory management abstracted away is helpful.

(c) Based on this declaration, or otherwise, create a class Deck that can be used to represent and manipulate a deck of cards. The class should declare the necessary data members, and at least the following member functions:

(constructor) //create a new deck of cards in order
dealCard //returns a single card from the top of deck and erases the card from deck
cardsLeft //returns the number of cards left in the deck
printCard //displays the card in human readable form, e.g. (4,1) is "Ace of Spades"

[12] 1 for Access specifier; 1 for data members, 4 for constructor, 2 for dealcard, 1 for cardsleft, and 3 for printCard

Note that the random access iterator is used to access elements at an arbitrary offset position relative to the element it points to, i.e. the *numerical index* of a particular element is not the primary model of access to member data, `iterator` is. However, given exam conditions, no marks will be deducted for accessing the vector using `at` and `erase` as if it were numerical index rather than relative. Some students may be aware of/think that `[]` operator is defined for element access (which it is) and solutions using `[]` will be equally credited.

```
class Deck {
        //Access specifier
        public:

        //Data members
        int numCards = 0 ;
        vector <pair<int,int>> Cards ;

        //Constructor and Member functions
        Deck () {
        pair<int,int> card ;

                for (int i=0; i < numSuits; i++){ //iterate through suits
                        for (int j=0 ; j < numFaces; j++){ //iterate through faces
                                card = make_pair(i,j) ; //make a card
                                Cards.push_back(card);  // add card to vector
                                numCards++;// increment no. of cards in deck
                        }
                }
        }

        pair<int,int> dealCard () {
                pair<int,int> card = Cards.at(0) ;
                Cards.erase(0) ;
                numCards-- ;
                return card ;
        }

        int cardsLeft () {
                Return numCards ;
        }

        void printCard (pair<int,int> card) {
                switch card.second {
                        case 1: cout << "Ace" ;
                        case 2: cout << "2" ;
                        // etc
                        case 13: cout << "King" ;
                }
                cout << " of " ;
                switch card.first {
                        case 1: cout << "Hearts" ;
                        //etc
                }
                cout << endl ;
        }
```

(d) Create a class Player that could be used to represent a player and her hand. Write two member functions:

dealtCard // the player is dealt a card

playCard // the player plays a random card from her hand

Write some lines of code to show how a player is dealt and plays a card.
[6]

```
Class Player
{
        public:

        int numCards = 0 ;
        string name ;
        vector <pair<int,int>> Hand ;

        void dealtCard (pair<int,int> card)
        {
                Hand.push_back(card) ;
                numCards++ ;
        }

        pair<int,int> playCard ()
        {
                int rando = rand()%numCards ;
                card = Hand.at(rando) ;
                Hand.erase(Hand.begin() + rando) ;
                numCards-- ;
                return card ;
        }
}

Player north ;
Deck pack ;
pair<int,int> card ;

north.dealtCard(pack.dealCard) ;
card = north.playCard ;
```

(e) Write a member function shuffle for class Deck which would shuffle a deck of cards.
[6]

declare two decks, the one in order and the shuffled,
vector <pair<int,int>> Cards, Shuffled ;

then after making the standard deck:

```
void Shuffle () {
        int temp = numCards ; //remember how many cards
        while (numCards > 0){
                int rando = rand()%numCards ;
                Shuffled.push_back( Cards.at( rando) ) ;
                Cards.erase(Cards.begin() + rando) ;
                numCards-- ;
                }
        numCards = temp ; //restore numCards
}
```

(f) Modify the member function for class Player that would return a card of a given suit, or a random card if the player has no cards of that suit.
[6]

```
pair<int,int> playCard (int suit){
        int it = -1 ; //increment before test, zero-based, so start from -1
        bool found = false ; //haven't found card of same suit yet

        while (!found && it <= Hand.numCards) {
                it++ ;
                found = Hand.at(it).first == suit ;
        }

        if (found){
                card = Hand.at(it) ;
        else{
                it = rand()%numCards ;
                card = Hand.at(it) ;
        }

        Hand.erase( Hand.begin() + it ) ;
        Hand.numCards-- ;

        return card ;
}
```

(g) Internally, a vector uses a dynamically-allocated array to store its elements. Vector containers may allocate some extra storage to accommodate possible growth. Comment on the advantages to the programmer of the vector container, and discuss the trade-offs (between memory-use and processing time) involved in allowing for extra storage.
[4]

The advantage to the programmer is that she does not have to be concerned with the implementation details of memory management (i.e. abstraction).

A dynamically allocated array may have to be re-allocated if the size outgrows the memory allocated to it. This is 'expensive' in terms of processing time, and so using vectors with some additional capacity means this can be avoided each time an element is added to the container. However, the vector can still be accessed using offsets on regular pointers to its elements, and just as efficiently as in (static) arrays. This is 'cheap' in terms of processing time. Thus vectors consume more memory in exchange for the ability to manage storage and grow dynamically in an efficient way (in fact, using push_back, memory allocation should only occur at logarithmically growing intervals of size, so that the insertion of individual elements at the end of the vector can be provided with *amortized constant time* complexity).

3 You have been hired as a developer to join a team creating some application ShapeWorld in C++. It is essential that you implement your code for ShapeWorld according to the specification and instructions that you are given.

(a) Define a structure coord for representing an (x,y)-coordinate in a 2-dimensional plane. [2]

```
struct coord {
        int x ;
        int y ;
}
```

(b) Define an array for storing 4 (four) such coordinates. [2]

```
coord corners [4] ;
```

(c) Suppose that a rectangular shape for some application ShapeWorld requires that the shape's lower left corner should be under the upper left corner, the lower right under the bottom right, the upper right to the right of the upper left, and the lower right to the right of lower left.

Define a class Shape that can be used to store the four corners of such rectangles, assuming that the lower-left corner is stored in array element 0, upper-left in 1, upper-right in 2, and lower-right in 3. Your class should define two member functions setCorner(…) and getCorner(…) to access the coordinate of the corner corresponding to the given index. [4]

```
class Shape {
        private:
                coord corners [4] ;

        public:
                void setCorner (int idx, int a, int b) {
                        corners[idx].x = a ;
                        corners[idx].y = b ;
                }

                coord getCorner (int idx) {
                        return corners[idx] ;
                }
}
```

(d) Define a constructor function for Shape using only two (x,y) coordinates arguments. Give a code snippet using the constructor for creating two Shape objects. Explain why defining a constructor function is preferable to using setCorner().
[6]

```
Shape (coord, coord) ;

Shape::Shape (coord lowleft, coord upright) {
        corners[0].x = lowleft.x ;
        corners[0].y = lowleft.y ;
        corners[1].x = lowleft.x ;
        corners[1].y = upright.y;
        corners[2].x = upright.x ;
        corners[2].y = upright.y ;
        corners[3].x = upright.x ;
        corners[3].y = lowleft.y ;
}

coord lowerLeft = {1,2} ;
coord upperRight = {3,4} ;

Shape rect1 ( lowerLeft, upperRight ) ;
```

The constructor is an improvement because setCorner would have to be called 4 times; it is generally better to have a constructor initialise an object so that it must precede any call to a get access function; it improves data abstraction

(e) Although the constructor function creates a valid Shape object for this application, a programmer could still misuse setCorner() and create an invalid Shape object. Write a member function validShape that checks whether or not a Shape object is (still) valid.
[4]

```
bool validShape () {
        return  (corners[0].x == corners[1].x)        &&
                (corners[0].y < corners[1].y)         &&
                (corners[1].x < corners[2].x)         &&
                (corners[1].y == corners[2].y)        &&
                (corners[2].x == corners[3].x)        &&
                (corners[3].y > corners[3].y) ;
}
```

(f) Write a member function overlappingShape(…) that checks if a valid Shape object (returns true if the two shapes overlap). Unfortunately your client *absolutely insists* that this function is written by checking if any one of the corners of one shape is within the boundaries of the other. Give a code snippet to show how your member function would be called. [6]

```
bool between ( int a, int b, int c ) {
        return (a < b) && (b < c) ;
}

bool overlappingShape (Shape otherShape) {
        int i ;
        coord corner ;
        bool olap = false ; //assume it does not overlap

        for (i=0; i<4; i++) {
                corner = otherShape.getCorner( i ) ;
                olap = olap || ((between(corners[0].x, corner.x, corners[2].x))
                                && (between(corners[0].y, corner.y, corners[2].y))) ;
        }
        return olap ;
}

coord lowerLeft1 = {1,2} ;
coord upperRight1 = {3,4} ;
coord lowerLeft2 = {5,6} ;
coord upperRight2 = {7,8} ;
bool safe ;

Shape rect1 ( lowerLeft1, upperRight1 ) ;
Shape rect2 ( lowerLeft2, upperRight2 ) ;

safe = !rect1.overlappingShape( rect2 ) ;
```

(g) Write a member function socialDistance(…) that returns true if the corners of one valid Shape object are at least 2 units away from the corners of another. (It can be assumed that there is a constant MaxDist which defines the maximum possible distance between any two shapes.)
[6]

```
bool socialDistance (Shape otherShape) {
        int i, j ;
        int xDiff, yDiff ;
        double separation, closest ; //make sure separation of closest corner is > 2.0

        closest = MaxDist ;
        for (i=0; i<4; i++) {
                for (j=0; j<4; j++) {
                        xDiff is abs(corners[j].x, otherShape.getCorner( i ).x) ;
                        yDiff is abs(corners[j].x, otherShape.getCorner( i ).y) ;
                        separation = sqrt((xDiff*xDiff) + (yDiff*yDiff)) ;
                        if (separation < closest) {
                                closest = separation ;
                        }
                }
        }
        return (closest > 2.0)  //however much one should not compare floats
                && !overlapping( otherShape ) ; //and shapes do not overlap
}
```